

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Выполнил: А. В. Козлов
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов

Москва, 2025

Условие

Цель работы:

Приобретение практических навыков в:

- Освоение принципов работы с файловыми системами
- Обеспечение обмена данных между процессами посредством технологии «File mapping»

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. **Вариант: 6**

Метод решения

Используется разделяемая память через memory-mapped files для обмена данными между родительским и дочерним процессами. Создается структура `shared_data_t`, содержащая буфер для данных, флаг готовности данных и флаг остановки. Реализована модель producer-consumer с использованием флагов синхронизации. Родительский процесс читает строки из стандартного ввода и записывает их в разделяемую память, устанавливая флаг `data_ready`. Дочерний процесс ожидает установки флага `data_ready`, проверяет начинается ли строка с заглавной буквы, и если да - записывает ее в файл. После обработки данных дочерний процесс сбрасывает флаг `data_ready`. Для избежания активного ожидания используется `usleep(1000)`. Когда ввод завершается, родительский процесс устанавливает флаг `stop` для корректного завершения дочернего процесса.

Описание программы

Программа состоит из двух процессов - родительского и дочернего, которые взаимодействуют через разделяемую память. Родительский процесс читает строки из стандартного ввода и передает их дочернему процессу через общую память. Дочерний процесс проверяет, начинается ли каждая полученная строка с заглавной буквы, и если да - записывает ее в указанный файл, а если нет - выводит сообщение об ошибке.

Родительский процесс создает файл разделяемой памяти с помощью memory-mapped files, где размещается структура данных для обмена. Затем создается дочерний процесс через `fork()`, который запускает программу `child` для обработки данных. Родительский процесс читает ввод пользователя, копирует строки в разделяемую память и синхронизируется с дочерним процессом через флаги `data_ready` и `stop`.

Дочерний процесс открывает ту же область разделяемой памяти, отслеживает появление новых данных и выполняет проверку строк на заглавную букву. Программа корректно завершает работу при получении сигнала остановки, освобождая все занятые ресурсы.

Результаты

Input:

`./parent`

Введите имя файла для результатов: `output.txt`

`parent proc`

`child proc`

Hello World
Привет мир
Test String
Строка не начинается с заглавной буквы: Привет мир

Another line
Exit
Родительский процесс завершен.

Output:
Hello World
Test String
Another line

Выводы

Разработанная программа успешно реализует межпроцессное взаимодействие через разделяемую память. Модель producer-consumer обеспечивает надежную передачу данных между родительским и дочерним процессами.

Архитектура демонстрирует стабильную работу с различными входными данными. Фильтрация строк по критерию первой заглавной буквы выполняется корректно для текста на разных языках.

Программа правильно управляет ресурсами разделяемой памяти и файловыми дескрипторами, что гарантирует отсутствие утечек памяти и корректное завершение работы обоих процессов. Синхронизация через флаги data_ready и stop обеспечивает согласованное взаимодействие процессов.

Исходная программа

```
1  #pragma once
2
3  #ifdef _WIN32
4
5  #else
6  #include <errno.h>
7  #include <fcntl.h>
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <string.h>
11 #include <sys/mman.h>
12 #include <sys/stat.h>
13 #include <sys/wait.h>
14 #include <unistd.h>
15
16 typedef int pid_t;
17
18 typedef struct {
19     void* address;
20     size_t size;
21 } mapped_file_t;
22 #endif
23
24 typedef int pid_t;
25
26 pid_t CloneProcess();
27
28 int Exec(const char* path, char* argv[]);
29
30 int WaitProcess();
31
32 mapped_file_t CreateMappedFile(const char* filename, size_t size);
33 void* OpenMappedFile(const char* filename, size_t size);
34 int CloseMappedFile(mapped_file_t mf);
35 int SyncMappedFile(mapped_file_t mf);
```

Листинг 1: *Интерфейс вызовов*

```
1  #include "os.h"
2
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <unistd.h>
7  #include <sys/wait.h>
8
9  pid_t CloneProcess() {
10     return fork();
11 }
12
13 int Exec(const char* path, char* argv[]) {
14     return execv(path, argv);
15 }
16
17 int WaitProcess() {
```

```

18     return wait(NULL);
19 }
20
21 mapped_file_t CreateMappedFile(const char* filename, size_t size) {
22     mapped_file_t result = {NULL, 0};
23     int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0666);
24     if (fd == -1) {
25         perror("open file error");
26         return result;
27     }
28     if (ftruncate(fd, size) == -1) {
29         perror("ftruncate error");
30         close(fd);
31         return result;
32     }
33     void* addr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
34     if (addr == MAP_FAILED) {
35         perror("mmap error");
36         close(fd);
37         return result;
38     }
39     close(fd);
40     result.address = addr;
41     result.size = size;
42     return result;
43 }
44
45 void* OpenMappedFile(const char* filename, size_t size) {
46     int fd = open(filename, O_RDWR);
47     if (fd == -1) {
48         perror("open file error");
49         return NULL;
50     }
51
52     void* addr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
53     close(fd);
54
55     if (addr == MAP_FAILED) {
56         perror("mmap error");
57         return NULL;
58     }
59
60     return addr;
61 }
62
63 int CloseMappedFile(mapped_file_t mf) {
64     if (mf.address && mf.size > 0) {
65         return munmap(mf.address, mf.size);
66     }
67     return -1;
68 }
69
70 int SyncMappedFile(mapped_file_t mf) {
71     if (mf.address && mf.size > 0) {
72         return msync(mf.address, mf.size, MS_SYNC);
73     }
74     return -1;

```

Листинг 2: *Реализация вызовов*

```

1  #include "os.h"
2
3  #include <signal.h>
4  #include <iostream>
5
6  #include "os.h"
7
8  #define BUFFER_SIZE 256
9  #define SHARED_MEMORY_SIZE 4096
10
11 typedef struct {
12     char data[BUFFER_SIZE];
13     int data_ready;
14     int stop;
15 } shared_data_t;
16
17 int main() {
18     pid_t pid;
19     char filename[BUFFER_SIZE];
20     mapped_file_t shared_mem;
21
22     printf("      : ");
23     if (!fgets(filename, BUFFER_SIZE, stdin)) {
24         perror("fgets error");
25         exit(1);
26     }
27     filename[strcspn(filename, "\n")] = 0;
28
29     shared_mem = CreateMappedFile("/tmp/shared_memory", sizeof(shared_data_t));
30     if (shared_mem.address == NULL) {
31         perror("CreateMappedFile error");
32         exit(1);
33     }
34
35     shared_data_t* shared_data = (shared_data_t*)shared_mem.address;
36     memset(shared_data, 0, sizeof(shared_data_t));
37
38     pid = CloneProcess();
39     if (pid == -1) {
40         perror("fork error");
41         CloseMappedFile(shared_mem);
42         exit(1);
43     }
44
45     if (pid == 0) {
46         std::cout << "child proc\n";
47         CloseMappedFile(shared_mem);
48         char* argv[] = {"child", filename, NULL};
49         Exec("./child", argv);
50         perror("exec error");
51         exit(1);
52     } else {
53         std::cout << "parent proc\n";

```

```

54     char buffer[BUFFER_SIZE];
55     FILE* output_file = fopen(filename, "w");
56     if (output_file == NULL) {
57         perror("fopen error");
58         CloseMappedFile(shared_mem);
59         exit(1);
60     }
61     while (1) {
62         if (fgets(buffer, BUFFER_SIZE, stdin) == NULL) {
63             break;
64         }
65         strncpy(shared_data->data, buffer, BUFFER_SIZE - 1);
66         shared_data->data_ready = 1;
67         while (shared_data->data_ready == 1) {
68             usleep(1000);
69         }
70
71         if (shared_data->stop) {
72             break;
73         }
74     }
75     shared_data->stop = 1;
76     shared_data->data_ready = 1;
77
78     fclose(output_file);
79     CloseMappedFile(shared_mem);
80     WaitProcess();
81     printf("  .\n");
82 }
83 }

```

Листинг 3: *Родительский процесс*

```

1  #include "os.h"
2  #include <stdio.h>
3  #include <iostream>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <ctype.h>
7
8  #define BUFFER_SIZE 256
9
10 typedef struct {
11     char data[BUFFER_SIZE];
12     int data_ready;
13     int stop;
14 } shared_data_t;
15
16 int main(int argc, char* argv[]) {
17     if (argc < 2) {
18         std::cerr << "fail" << argv[0] << "\n";
19         exit(1);
20     }
21     void* shared_addr = OpenMappedFile("/tmp/shared_memory", sizeof(shared_data_t));
22     if (shared_addr == nullptr) {
23         perror("OpenMappedFile error");
24         exit(1);

```

```

25     }
26     shared_data_t* shared_data = (shared_data_t*)shared_addr;
27     const char* output_filename = argv[1];
28     while (1) {
29         while (shared_data->data_ready == 0 && !shared_data->stop) {
30             usleep(1000);
31         }
32         if (shared_data->stop) {
33             break;
34         }
35         char* line = shared_data->data;
36         FILE* output_file = fopen(output_filename, "a");
37         if (output_file == NULL) {
38             perror("fopen error");
39             break;
40         }
41         if (isupper((unsigned char)line[0])) {
42             fprintf(output_file, "%s", line);
43             fflush(output_file);
44         } else {
45             std::cout << "      : " << line << "\n";
46         }
47
48         fclose(output_file);
49         shared_data->data_ready = 0;
50     }
51
52     CloseMappedFile((mapped_file_t){shared_addr, sizeof(shared_data_t)});
53     return 0;
54 }

```

Листинг 4: *Дочерний процесс*

```

1  #include <iostream>
2  #include <chrono>
3
4  #include "matrix.hpp"
5  #include "thread.hpp"
6
7  int main(int argc, char* argv[]) {
8      if(argc < 2) {
9          std::cerr << "not enough args, needed threads";
10         return 1;
11     }
12     int threads_count = std::stoi(argv[1]);
13     int rows_a;
14     int cols_a;
15     int rows_b;
16     int cols_b;
17
18     std::cout << "      \n";
19     std::cin >> rows_a >> cols_a;
20     std::cout << "      \n";
21     std::cin >> rows_b >> cols_b;
22     Matrix A = create_matrix(rows_a, cols_a);
23     Matrix B = create_matrix(rows_b, cols_b);
24

```

```

25 |     auto begin = std::chrono::steady_clock::now();
26 |     Matrix result = multiply_matrices(A, B, threads_count);
27 |     auto end = std::chrono::steady_clock::now();
28 |     auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - begin)
    |         .count();
29 |     std::cout << duration << "ms\n";
30 | }

```

Листинг 5: *Основной файл программы*

```

    execve("./parent",["./parent"],0x7ffc164ff950 /* 49 vars */) = 0
    brk(NULL) = 0x5b59d3c4d000
    mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x7f74b4ed7000
    access("/etc/ld.so.preload",R_OK) = -1 ENOENT (No such file or
directory)
    openat(AT_FDCWD,"/etc/ld.so.cache",O_RDONLY|O_CLOEXEC) = 3
    fstat(3,{st_mode=S_IFREG|0644,st_size=60107,...}) = 0
    mmap(NULL,60107,PROT_READ,MAP_PRIVATE,3,0) = 0x7f74b4ec8000
    close(3) = 0

    openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libstdc++.so.6",O_RDONLY|O_CLOEXEC)
= 3

    read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
    fstat(3,{st_mode=S_IFREG|0644,st_size=2592224,...}) = 0
    mmap(NULL,2609472,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f74b4c00000

    mmap(0x7f74b4c9d000,1343488,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x7f74b4c9d000

    mmap(0x7f74b4de5000,552960,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1e5000)
= 0x7f74b4de5000

    mmap(0x7f74b4e6c000,57344,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x7f74b4e6c000

    mmap(0x7f74b4e7a000,12608,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,
= 0x7f74b4e7a000
    close(3) = 0

    openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3

    read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832

    pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)

```

```

= 784
    fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
    mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f74b4800000

mmap(0x7f74b4828000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b4828000

mmap(0x7f74b49b0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)
= 0x7f74b49b0000

mmap(0x7f74b49ff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b49ff000

mmap(0x7f74b4a05000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,0)
= 0x7f74b4a05000
    close(3) = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libm.so.6",O_RDONLY|O_CLOEXEC) = 3

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
    fstat(3,{st_mode=S_IFREG|0644,st_size=952616,...}) = 0
    mmap(NULL,950296,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f74b4b17000

mmap(0x7f74b4b27000,520192,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b4b27000

mmap(0x7f74b4ba6000,360448,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x8f000)
= 0x7f74b4ba6000

mmap(0x7f74b4bfe000,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b4bfe000
    close(3) = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libgcc_s.so.1",O_RDONLY|O_CLOEXEC)
= 3

read(3,"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
    fstat(3,{st_mode=S_IFREG|0644,st_size=183024,...}) = 0
    mmap(NULL,185256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f74b4e9a000

```

```

mmap(0x7f74b4e9e000,147456,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b4e9e000

mmap(0x7f74b4ec2000,16384,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x28000)
= 0x7f74b4ec2000

mmap(0x7f74b4ec6000,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f74b4ec6000
    close(3) = 0
    mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x7f74b4e98000
    arch_prctl(ARCH_SET_FS,0x7f74b4e99500) = 0
    set_tid_address(0x7f74b4e997d0) = 5759
    set_robust_list(0x7f74b4e997e0,24) = 0
    rseq(0x7f74b4e99e20,0x20,0,0x53053053) = 0
    mprotect(0x7f74b49ff000,16384,PROT_READ) = 0
    mprotect(0x7f74b4ec6000,4096,PROT_READ) = 0
    mprotect(0x7f74b4bfe000,4096,PROT_READ) = 0
    mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x7f74b4e96000
    mprotect(0x7f74b4e6c000,45056,PROT_READ) = 0
    mprotect(0x5b59b3404000,4096,PROT_READ) = 0
    mprotect(0x7f74b4f15000,8192,PROT_READ) = 0

prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})
= 0
    munmap(0x7f74b4ec8000,60107) = 0
    futex(0x7f74b4e7a7bc,FUTEX_WAKE_PRIVATE,2147483647) = 0
    getrandom("\x7c\x4c\xe1\x28\x07\x7e\x09\x18",8,GRND_NONBLOCK) = 8
    brk(NULL) = 0x5b59d3c4d000
    brk(0x5b59d3c6e000) = 0x5b59d3c6e000
    fstat(1,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0),...}) = 0
    fstat(0,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0),...}) = 0
    write(1,"\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260"... ,64)
= 64
    read(0,"1.txt\n",1024) = 6
    openat(AT_FDCWD,"/tmp/shared_memory",O_RDWR|O_CREAT|O_TRUNC,0666) =
3
    ftruncate(3,264) = 0
    mmap(NULL,264,PROT_READ|PROT_WRITE,MAP_SHARED,3,0) = 0x7f74b4ed6000
    close(3) = 0

clone(child_stack=NULL,flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,child_tid
= 5760
    write(1,"parent proc\n",12) = 12
    openat(AT_FDCWD,"1.txt",O_WRONLY|O_CREAT|O_TRUNC,0666) = 3
    read(0,"Popa\n",1024) = 5

```

```

0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,"Pisa\n",1024) = 5
0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,"pisa\n",1024) = 5
0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,"hohoho\n",1024) = 7
0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,"AHAHA\n",1024) = 6
0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,"11\n",1024) = 3
0      clock_nanosleep(CLOCK_REALTIME,0,{tv_sec=0,tv_nsec=1000000},NULL) =
0      read(0,0x5b59d3c5f6c0,1024) = ? ERESTARTSYS (To be
restarted if SA_RESTART is set)
    ---SIGINT {si_signo=SIGINT,si_code=SI_KERNEL} ---
    +++ killed by SIGINT +++

```

Листинг 6: *Strace логи*